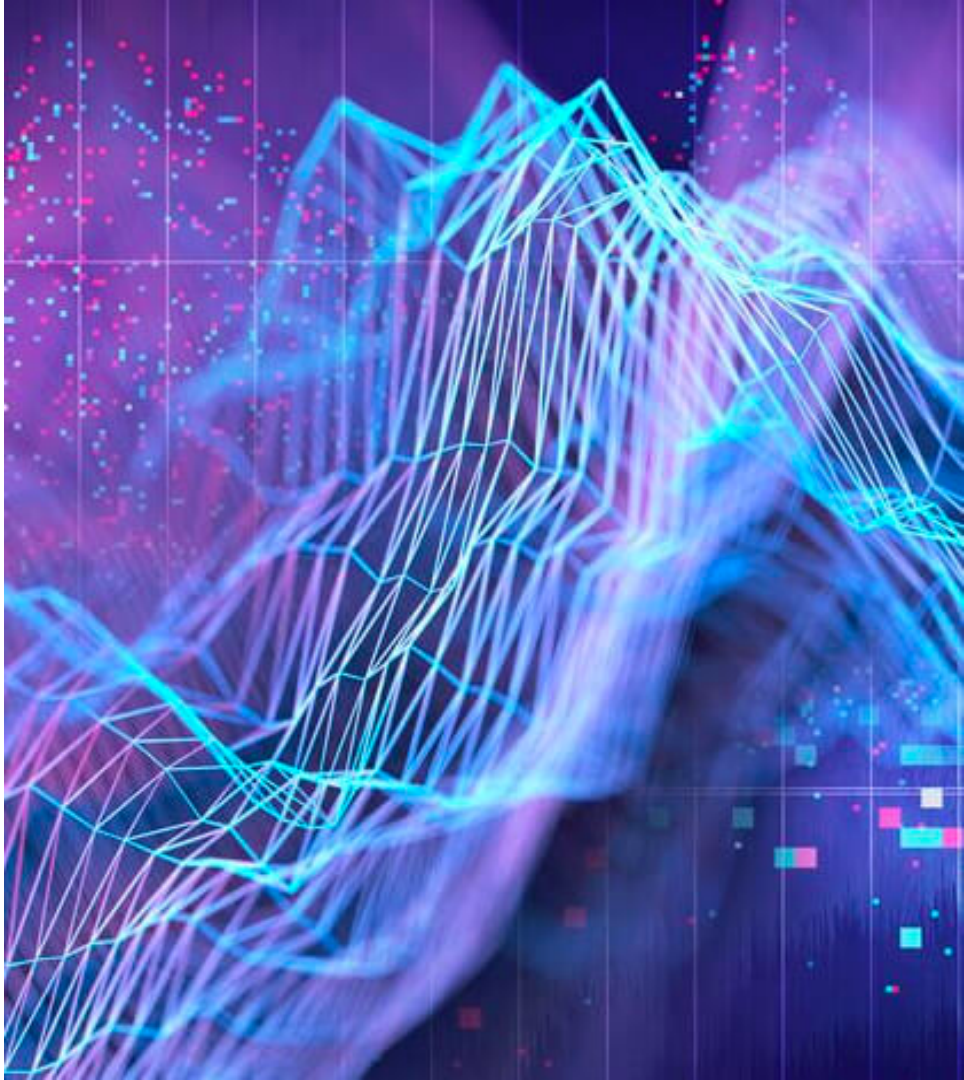
Curse of Dimensionality, Probability and Naive Bayes

Machine Learning for Electronics Engineers

Shivin Srivastava

Slides Adapted from CS441 UIUC



Outline for This Week

1. Curse of Dimensionality

2. Maximum Likelihood Estimation (MLE)

3. Maximum a posteriori Probability (MAP)

4. Example: MLE and MAP for classification

5. The Naive Bayes Classifier

The K-NN Algorithm

Input: classification training dataset $\{x_i, y_i\}_{i=1}^n$, and parameter $K \in \mathbb{N}^+$, and a distance metric $d(x, x')$ (e.g., $\|x - x'\|_2$ euclidean distance)

K-NN Algorithm:

Store all training data

For any test point x :

> Find its top K nearest neighbors (under metric d)
>
> Return the most common label among these K neighbors
>
> (If for regression, return the average value of the K neighbors)

The choice of K

1. What if we set K very large?

Top K-neighbors will include examples that are very far away…

2. What if we set K very small (K=1)?

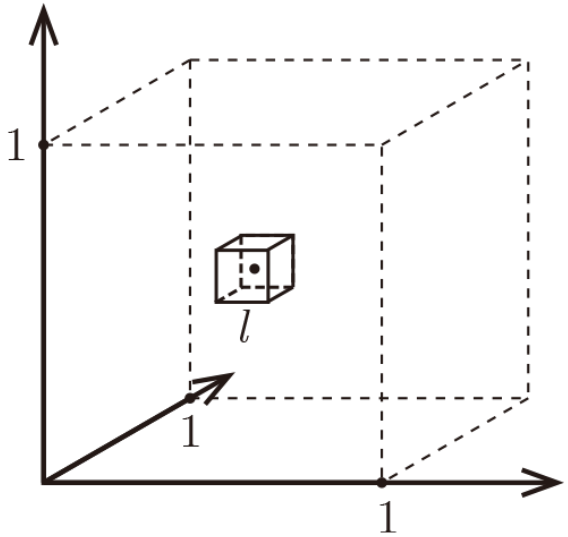
label has noise (easily overfit to the noise)

(What about the training error when K = 1?)

Curse of Dimensionality Explanation

Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0,1]^d$

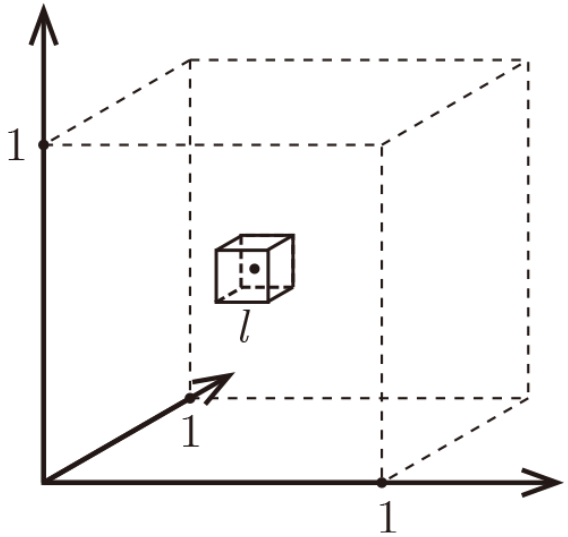


Q: sample x uniformly, what is the probability that x is inside the small cube?

Curse of Dimensionality Explanation

Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0,1]^d$



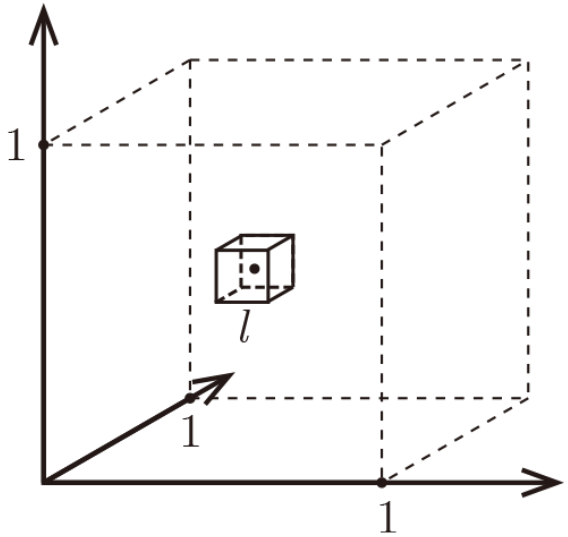
Q: sample x uniformly, what is the probability that x is inside the small cube?

A: Volume(small cube)/volume($[0,1]^d$)

Curse of Dimensionality Explanation

Key problem: in high dimensional space, points that are draw from a distribution tends to be far away from each other!

Example: let us consider uniform distribution over a cube $[0,1]^d$

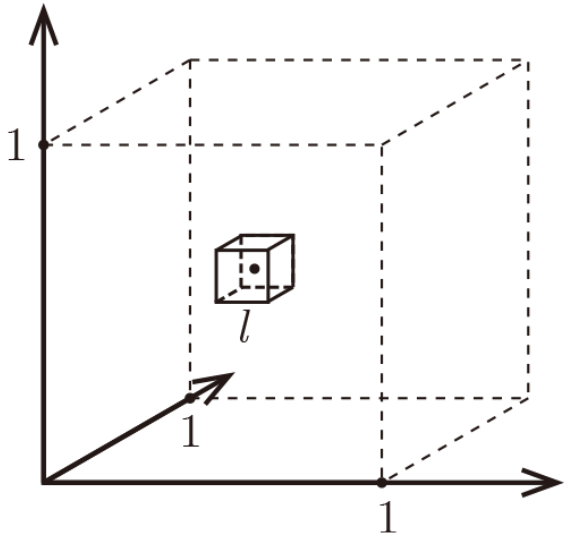


Q: sample x uniformly, what is the probability that x is inside the small cube?

A: Volume(small cube)/volume($[0,1]^d$) $= l^d$

Curse of Dimensionality Explanation

Example: let us consider uniform distribution over a cube $[0,1]^d$



Now assume we sample n points uniform randomly, and we observe K points fall inside the small cube

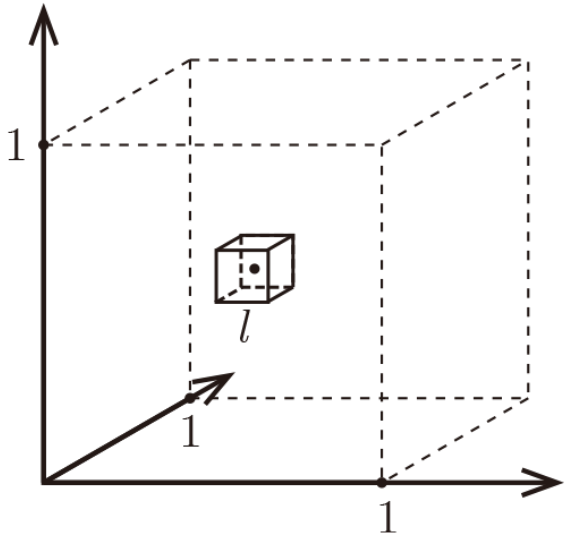
So empirically, the probability of sampling a point inside the small cube is roughly K/n

Thus, we have $l^d \approx \frac{K}{n}$

Curse of Dimensionality Explanation

Example: let us consider uniform distribution over a cube $[0,1]^d$

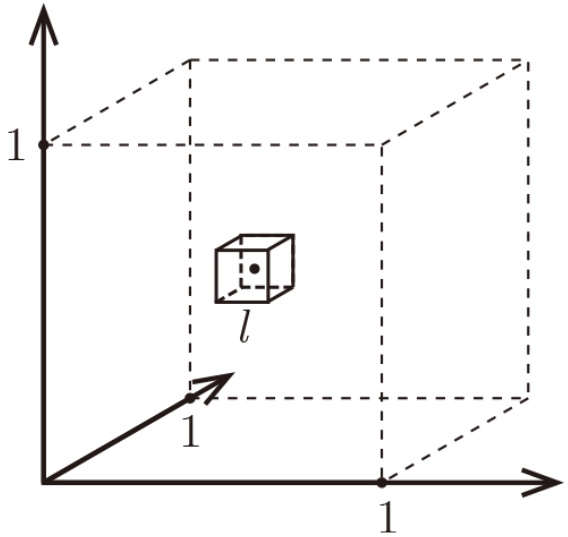
We have $l^d \approx \dfrac{K}{n}$



Curse of Dimensionality Explanation

Example: let us consider uniform distribution over a cube $[0,1]^d$

We have $l^d \approx \frac{K}{n}$

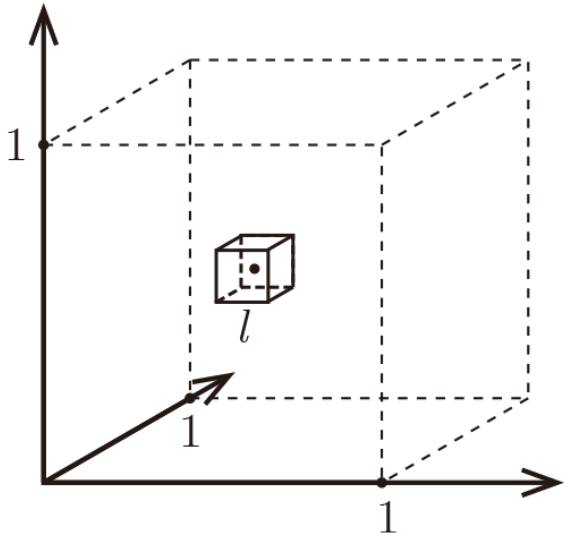


Q: how large we should set l, s.t., we will have K examples (out of n) fall inside the small cube?

Curse of Dimensionality Explanation

Example: let us consider uniform distribution over a cube $[0,1]^d$

We have $l^d \approx \frac{K}{n}$



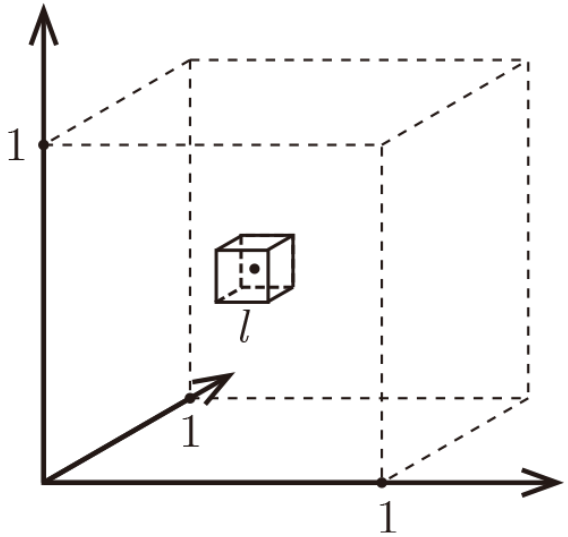
Q: how large we should set l, s.t., we will have K examples (out of n) fall inside the small cube?

$$l \approx (K/n)^{1/d}$$

Curse of Dimensionality Explanation

Example: let us consider uniform distribution over a cube $[0,1]^d$

We have $l^d \approx \frac{K}{n}$



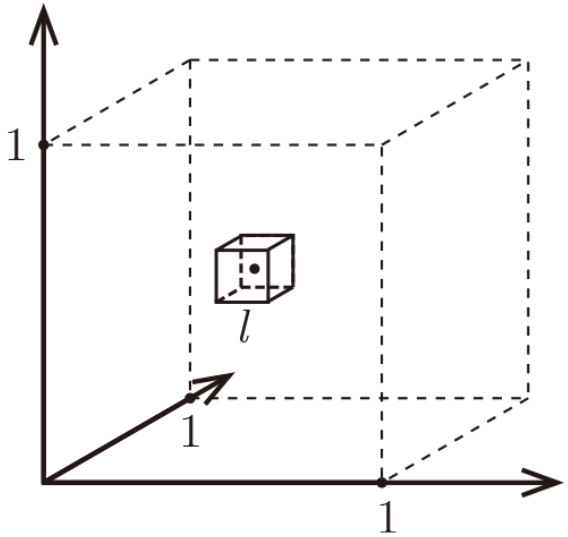
Q: how large we should set l, s.t., we will have K examples (out of n) fall inside the small cube?

$$l \approx (K/n)^{1/d} \to 1, \text{ as } d \to \infty$$

Curse of Dimensionality Explanation

Example: let us consider uniform distribution over a cube $[0,1]^d$

We have $l^d \approx \frac{K}{n}$



Q: how large we should set l, s.t., we will have K examples (out of n) fall inside the small cube?

$$l \approx (K/n)^{1/d} \to 1, \text{ as } d \to \infty$$

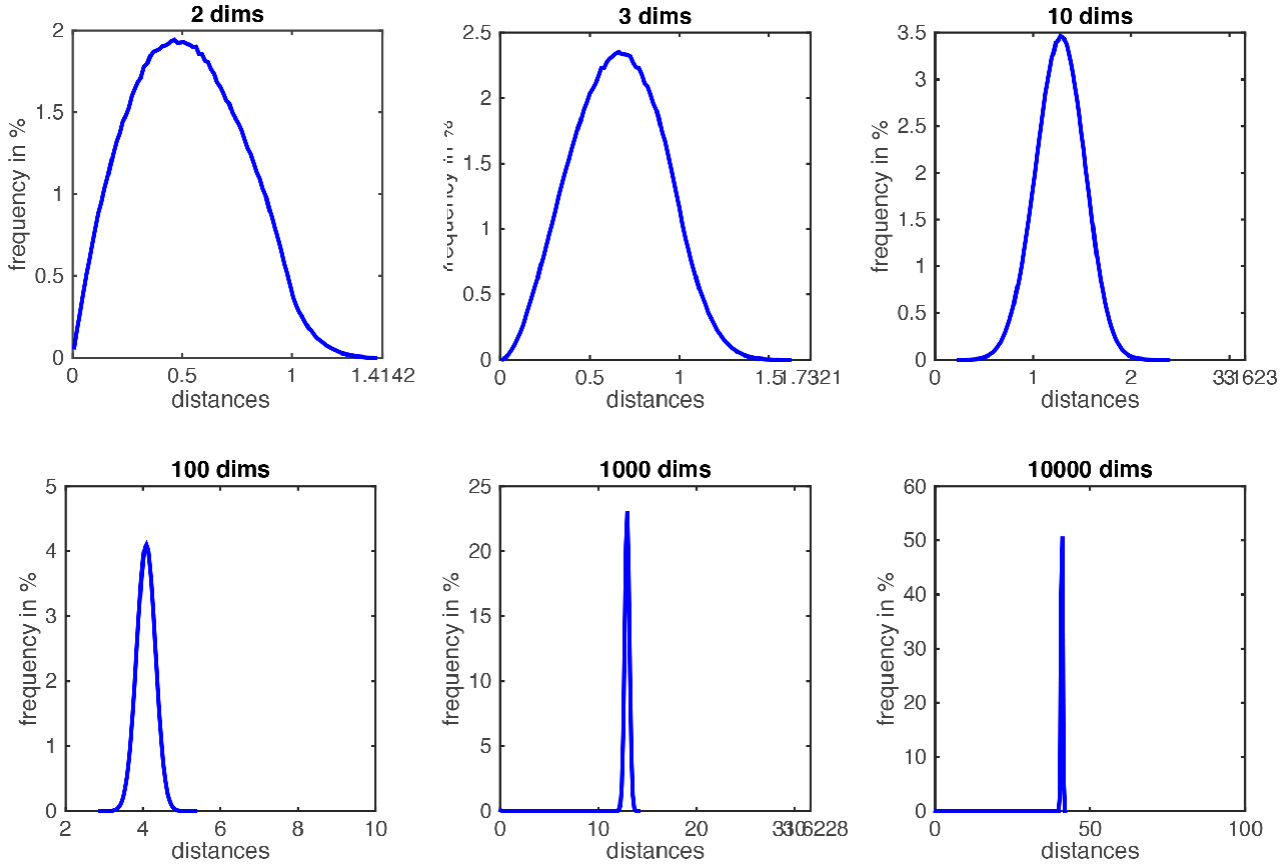
Bad news: when $d \to \infty$, the K nearest neighbors will be all over the place! (Cannot trust them, as they are not nearby points anymore!)

The distance between two sampled points increases as d grows

In $[0,1]^d$, we uniformly sample two points x, x', calculate

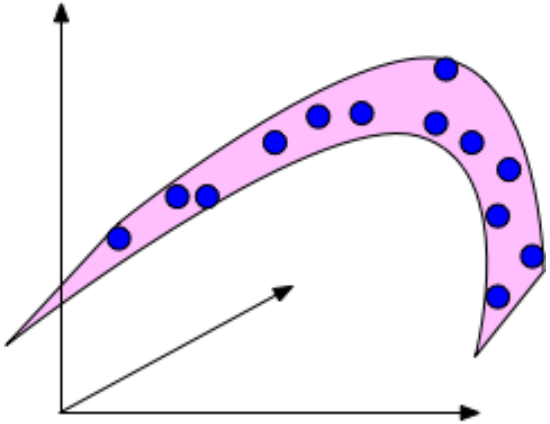
$$d(x, x') = \|x - x'\|_2$$

Let's plot the distribution of such distance:



Distance increases as $d \to \infty$

Luckily, real world data often has low-dimensional structure!



Data lives in 2-d manifold

Example: face images

Original image: $\mathbb{R}^{64^2}$

Outline for This Week

1. Curse of Dimensionality

2. The Naive Bayes Classifier

2. Maximum Likelihood Estimation (MLE)

3. Maximum a posteriori Probability (MAP)

4. Example: MLE and MAP for classification

Probabilistic model

$$y^* = \underset{y}{\operatorname{argmax}} P(y \mid x)$$

Joint and conditional probability

$$P(x, y) = P(x \mid y)P(y) = P(y \mid x)P(x)$$

$$P(a, b, c) = P(a \mid b, c)P(b \mid c)P(c)$$

Bayes Rule: $$P(x \mid y) = \frac{P(x, y)}{P(y)} = \frac{P(y \mid x)P(x)}{P(y)}$$

Probabilistic model

$$y^* = \underset{y}{\operatorname{argmax}} P(y \mid x)$$

Or equivalently…

$$y^* = \underset{y}{\operatorname{argmax}} P\left(x \middle| y\right) P\left(y\right)$$

$$\underset{y}{\operatorname{argmax}}\ P\left(y \middle| x\right) = \underset{y}{\operatorname{argmax}}\ P\left(y \middle| x\right) P(x) = \underset{y}{\operatorname{argmax}}\ P\left(y, x\right) = \underset{y}{\operatorname{argmax}} P\left(x \middle| y\right) P(y)$$

Example

x:Larger than 10 lbs?

y	F	T
Cat	15	25
Dog	5	40

$$P(y = Cat) =$$

$$P(y = Cat \mid x = F) =$$

$$P(x = F \mid y = Cat) =$$

Law of total probability $$\left[\sum_{v \in x} P(x = v)\right] = 1$$

Marginalization $$\left[\sum_{v \in x} P\big(x = v, y\big)\right] = P(y)$$

For continuous variables, replace sum over possible values with integral over domain

A is independent of B if (and only if)

$$P(A, B) = P(A)P(B)$$

$$P(A \mid B) = P(A), \quad P(B \mid A) = P(B)$$

Notation

- x_i is the ith feature variable
  - i indicates the feature index
- x_n is the nth feature vector
  - n indicates the sample index
  - y_n is the nth label
- x_{ni} is the ith feature of the nth sample
- $\delta(x_{ni} = v)$ returns 1 if $x_{ni} = v$; 0 otherwise
  - v indicates a feature value
  - δ is an indicator function, mapping from true/false to 1/0

Estimate probabilities of discrete variables by counting

$$P(x = v) = \frac{1}{|N|} \sum_{n} \delta\left(x_n = v\right)$$

What if you have 100 variables? How can you count all combinations?

Fully modeling dependencies between many variables (more than 3 or 4) is challenging and requires a lot of data

Naïve Bayes Model

Assume features $x_1..x_m$ are independent given the label y:

$$P(\boldsymbol{x} \mid y) = \prod_i P(x_i \mid y)$$

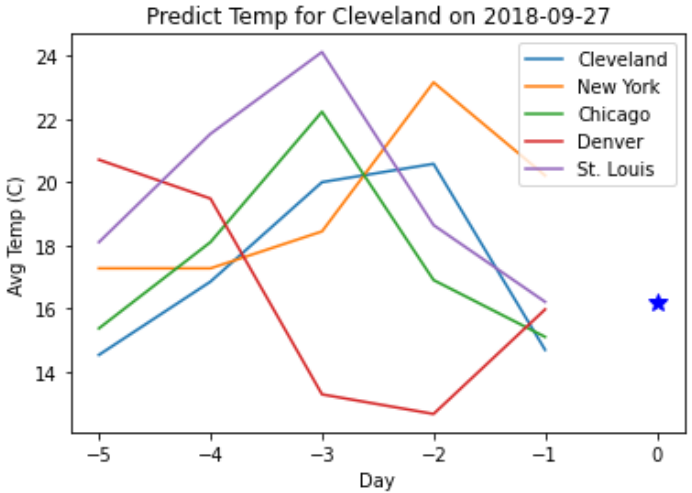
Then

$$y^* = \underset{y}{\operatorname{argmax}} \prod_i P(x_i \mid y) P(y)$$

Examples

- Digit classification: choose the label that maximizes the product of likelihoods of each pixel intensity

- Temperature prediction: each feature predicts y with some offset and variance ($y-x_i$ is univariate Gaussian)



Naïve Bayes Algorithm

- Training
  1. Estimate parameters for $P(x_i|y)$ for each i
  2. Estimate parameters for P(y)
- Prediction
  1. Solve for y that maximizes P(x,y)

$$y^* = \underset{y}{\operatorname{argmax}} \prod_i P\left(x_i \middle| y\right) P(y)$$

How to estimate $P(x_i|y)$ from data?

- Basic principles of fitting likelihood parameters from data
  - MLE (maximum likelihood estimation): Choose the parameter that maximizes the likelihood of the data
  - MAP (maximum a priori): Choose the parameter that maximizes the data likelihood and its own prior
    - As Warren Buffet says, it's not just about maximizing expected return – it's about making sure there are no zeros.

How to estimate $P(x_i|y)$ from data?

- Binomial (x is binary; y is discrete)

$$P(x_i \mid y = k) = \theta_{ki}^{x_i}(1 - \theta_{ki})^{1-x_i}$$

$$\theta_{ki} = \sum_n \delta(x_{ni} = 1, y_n = k) \Big/ \sum_n \delta(y_n = k)$$

```
theta_ki[k,i] = np.sum((X[:,i]==1) & (y==k)) / np.sum(y==k)
```

- Multinomial (x is has multiple discrete values, y is discrete)

$$\theta_{kiv} = \sum_n \delta(x_{ni} = v, y_n = k) \Big/ \sum_n \delta(y_n = k)$$

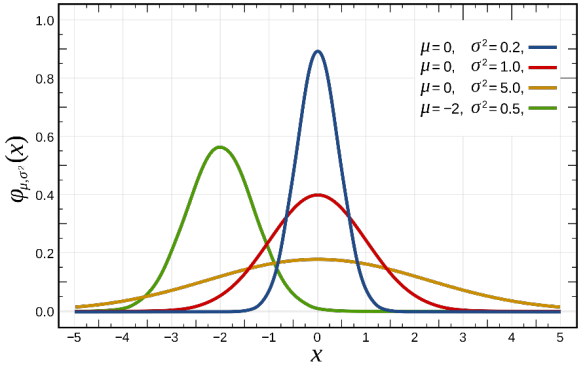
How to estimate $P(x_i|y)$ from data?

- x_i is Gaussian (aka Normal), y is discrete

$$P(x_i|y=k) = \frac{1}{\sqrt{2\pi}\,\sigma_{ki}} \cdot \exp\left(-\frac{1}{2}\frac{(x_i-\mu_{ki})^2}{\sigma_{ki}^2}\right)$$

$$\mu_{ki} = \sum_n \left[x_{ni}\cdot\delta(y_n=k)\right] \Big/ \sum_n \delta(y_n=k)$$

$$\sigma_{ki}^2 = \sum_n \left[(x_{ni}-\mu_{ki})^2\cdot\delta(y_n=k)\right] \Big/ \sum_n \delta(y_n=k)$$



How to estimate $P(x_i|y)$ from data?

- $(y-x_i)$ is Gaussian

$$P(y-x_i) = \frac{1}{\sqrt{2\pi}\,\sigma_i} \exp\left(-\frac{1}{2}\frac{(y-x_i-\mu_i)^2}{\sigma_i^2}\right)$$

$$\mu_i = \sum_n^N (y-x_i)/N$$

$$\sigma_i^2 = \sum_n^N (y-x_i-\mu_i)^2/N$$

```
mu[i] = np.mean(y-X[:,i], axis=0)
std[i] = np.std(y-X[:,i], axis=0)
```

How to estimate $P(x_i|y)$ from data?

- x_i and y are jointly Gaussian

$$P(x_i|y) = N([x_i, y]; \underline{\mu}_i, \underline{\Sigma}_i) / N(y; \mu_y, \sigma_y)$$

  – N(.) stands for normal distribution with given value, mean, and covariance or variance

How to estimate $P(x_i|y)$ from data?

- x_i is continuous (non-Gaussian), y is discrete
  - First turn x into discrete (e.g. if values range [0, 1), assign x=floor(x*10)
  - Now can estimate as multinomial

How to estimate $P(x_i|y)$ from data?

- If x is text, e.g. “blue”, “orange”, “green”
  - Map each possible text value into an integer and solve as multinomial

How to estimate P(y)?

Three options:

- Assume that y is “uniform” (every value is equally likely) and ignore
- If y is discrete, count
- If y is continuous, model as Gaussian or convert to discrete and count

Stretch break: Simple Naive Bayes example

- Suppose I want to classify a fruit based on description
  - Features: weight, color, shape, whether it's hard
  - E.g.
    - 0.5 lb, "red", "round", yes
    - 15 lb, "green", "oval", yes
    - 0.01 lb, "purple", "round", no

Q1: What are these three fruit?

Q2: How might you model $P(x_i|\text{fruit})$ for each of these four features?

Simple Naive Bayes example

- Suppose I want to classify a fruit based on description
  - Features: weight, color, shape, whether it's hard
  - E.g.
    - 0.5 lb, "red", "round", yes — Apple
    - 15 lb, "green", "oval", yes — Watermelon
    - 0.01 lb, "purple", "round", no — Grape
  - Model P(weight | fruit) as a Gaussian
  - Model P(color | fruit) as a discrete distribution (multinomial
  - Model P(shape | fruit) as a multinomial
  - Model P(is_hard | fruit) as a Bernoulli (binary)

How to predict y from x?

$$y^* = \underset{y}{\operatorname{argmax}} \prod_i P(x_i \mid y)\, P(y)$$

$$= \underset{y}{\operatorname{argmax}} \sum_i \log P(x_i \mid y) + \log P(y)$$

If y is discrete:

1. Compute P(x,y) for each value of y
2. Choose value with maximum likelihood

Turning product into sum of logs is an important frequently used trick for argmax/argmin!

How to predict y from x?

If y is continuous,

$$\frac{\partial}{\partial y} \sum_i \log P(x_i|y) + \log P(y) = 0$$

General formulation (set partial derivative wrt y of log P(x,y) to 0)

$$\frac{\partial}{\partial y} \sum_i -\frac{1}{2}\frac{(y-x_i-\mu_i)^2}{\sigma_i^2} - \frac{1}{2}\frac{(y-\mu_y)^2}{\sigma_y^2} = 0$$

Example: $y-x_i$ is Gaussian (HW1)

$$\frac{\partial}{\partial y} \sum_i \frac{1}{2}\frac{-y^2}{\sigma_i^2} + \frac{yx_i}{\sigma_i^2} + \frac{y\mu_i}{\sigma_i^2} \quad \frac{-1}{2}\frac{y^2}{\sigma_y^2} + \frac{y\mu_y}{\sigma_y^2} = 0$$

$$\sum_i \left(\frac{-y}{\sigma_i^2} + \frac{x}{\sigma_i^2} + \frac{\mu_i}{\sigma_i^2}\right) - (y-\mu_y)/\sigma_y^2 = 0$$

$$y\left(\sum_i \frac{1}{\sigma_i^2} + \frac{1}{\sigma_y^2}\right) = \sum_i \frac{x+\mu_i}{\sigma_i^2} + \frac{\mu_y}{\sigma_y^2}$$

$$y = \frac{1}{\sum_i \frac{1}{\sigma_i^2} + \frac{1}{\sigma_y^2}} \cdot \left[\sum_i \frac{x+\mu_i}{\sigma_i^2} + \frac{\mu_y}{\sigma_y^2}\right]$$

$$y = \frac{1}{\sum_i w_i + w_y} \cdot \left[\sum_i (x+\mu_i) w_i + \mu_y \cdot w_y\right]$$

Prediction is weighted average of means, where weights are inverse variance

Using priors

- Priors on the likelihood parameters prevent a single feature from having zero or extremely low likelihood due to insufficient training data

- Discrete: initialize counts with α (e.g. $\alpha = 1$)

$$P(x_i=v|y=k) = (\alpha + \text{count}(x_i=v, y=k)) / \text{sum}_v[\alpha + \text{count}(x_i=v, y=k)]$$

```
theta_kiv[k,i,v] = (np.sum((X[:,i]==v) & (y==k))+alpha) / (np.sum(y==k)+alpha*num_v)
```

- Continuous: add some ϵ to the variance (e.g. $\epsilon = 0.1/N$)
  - For multivariate, add to diagonal of covariance

```
std[i] = np.std(y-X[:,i], axis=0)+np.sqrt(0.1/len(X))
```

Example

#	x1	x2	y
1	1	1	1
2	0	1	1
3	1	0	0
4	0	1	0
5	1	1	1
6	1	0	0
7	1	0	1
8	0	1	0

$P(x1 \mid y)$

x1	
0	
1	

$P(x2 \mid y)$

x2	
0	
1	

$P(y)$

$P(y, x1 = 1, x2 = 1) = \ ?$

Example

#	x1	x2	y
1	1	1	1
2	0	1	1
3	1	0	0
4	0	1	0
5	1	1	1
6	1	0	0
7	1	0	1
8	0	1	0

$P(x1 \mid y)$

x1		
0	2/4	1/4
1	2/4	3/4

$P(x2 \mid y)$

x2		
0	2/4	1/4
1	2/4	3/4

$P(y)$

2/4	2/4

$P(y, x1 = 1, x2 = 1) = \ ?$

Prior over parameters: initialize each count with α

$\alpha = 1$

#	x1	x2	y
1	1	1	1
2	0	1	1
3	1	0	0
4	0	1	0
5	1	1	1
6	1	0	0
7	1	0	1
8	0	1	0

$P(x1 \mid y)$

x1		
0	2/4	1/4
1	2/4	3/4

x1		
0	3/6	2/6
1	3/6	4/6

$P(x2 \mid y)$

x2		
0	2/4	1/4
1	2/4	3/4

x2		
0	3/6	2/6
1	3/6	4/6

$P(y)$

2/4	2/4

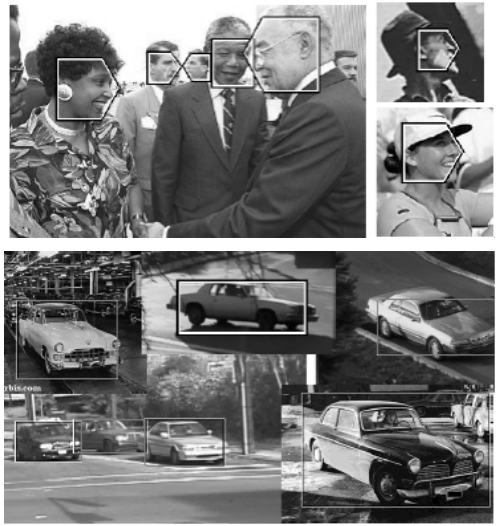
2/4	2/4

Use case: “Semi-naïve Bayes” object detection

- Best performing face/car detector in 2000-2005
- Model probabilities of small groups of features (wavelet coefficients)
- Search for groupings, discretize features, estimate parameters

A Statistical Method for 3D Object Detection Applied to Faces and Cars

Henry Schneiderman and Takeo Kanade



$$\frac{\prod_{x,y \in \text{region}} \prod_{k=1}^{17} P_k(pattern_k(x,y), x, y \mid \text{object})}{\prod_{x,y \in \text{region}} \prod_{k=1}^{17} P_k(pattern_k(x,y), x, y \mid \text{non-object})} > \lambda$$

https://www.cs.cmu.edu/afs/cs.cmu.edu/user/hws/www/CVPR00.pdf

Naïve Bayes Summary

- Key Assumptions
  - Features are independent, given the labels
- Model Parameters
  - Parameters of probability functions $P(x_i|y)$ and $P(y)$
- Designs
  - Choice of probability function
- When to Use
  - Limited training data
  - Features are not highly interdependent
  - Want something fast to code, train, and test
- When Not to Use
  - Logistic or linear regression will usually work better if there is sufficient data (more flexible / fewer assumptions than Naïve Bayes)
  - Does not provide a good confidence estimate because it “overcounts” influence of dependent variables

Naïve Bayes

- Pros
  - Easy and fast to train
  - Fast inference
  - Can be used with continuous, discrete, or mixed features
- Cons
  - Does not account for feature interactions
  - Does not provide good confidence estimate
- Notes
  - Best when used with discrete variables, variables that are well fit by Gaussian, or kernel density estimation

Things to remember

- Probabilistic models are a large class of machine learning methods
- Naïve Bayes assumes that features are independent given the label
  - Easy/fast to estimate parameters
  - Less risk of overfitting when data is limited
- You can look up how to estimate parameters for most common probability models
  - Or take partial derivative of total data/label likelihood given parameter
- Prediction involves finding y that maximizes P(x,y), either by trying all y or solving partial derivative
- Maximizing log P(x,y) is equivalent to maximizing P(x,y) and often much easier

$$P(\boldsymbol{x}, y) = \prod_i P(x_i \mid y) P(y)$$

$$y^* = \underset{y}{\operatorname{argmax}} \prod_i P(x_i \mid y) P(y)$$

$$= \underset{y}{\operatorname{argmax}} \sum_i \log P(x_i \mid y) + \log P(y)$$

Thank You!

Any Questions?